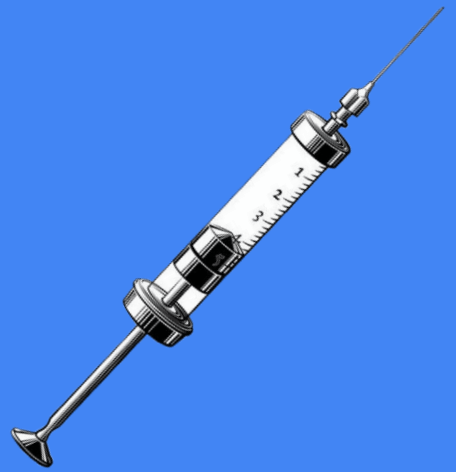
Dependency injection

Don't let dependencies drag you down. Inject them up!

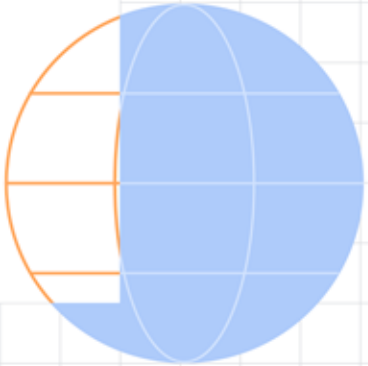


Content

- Intro
- Manual DI implementation
- Intro to Koin
- Koin implementation
- Koin additional features
- Best practices & common pitfalls
- Alternatives
- Conclusion

GDG

Intro



Intro

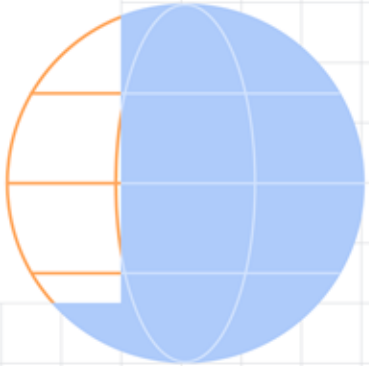
- Dependency – **direct relationship between two classes**; i.e. one relies on another for its functionality
- Dependency injection – implementation technique for **populating instance variables of a class**
- Based onto **Dependency Inversion Principle** (SOLI**D**) – high-level modules shouldn't depend on low-level modules, both **should depend on abstractions**

Intro

- 4 roles of DI:
  - **Service** – class that carries some functionality
  - **Client** – class that requests something from a service
  - **Interface** – abstract component implemented by a service for use by one or more clients
  - **Injector** – component that introduces a service to a client; i.e. creating and inserting a service into a client
- Types of injection:
  - **Constructor injection** (clear glance on required dependencies)
  - Public property/field injection
  - Method injection

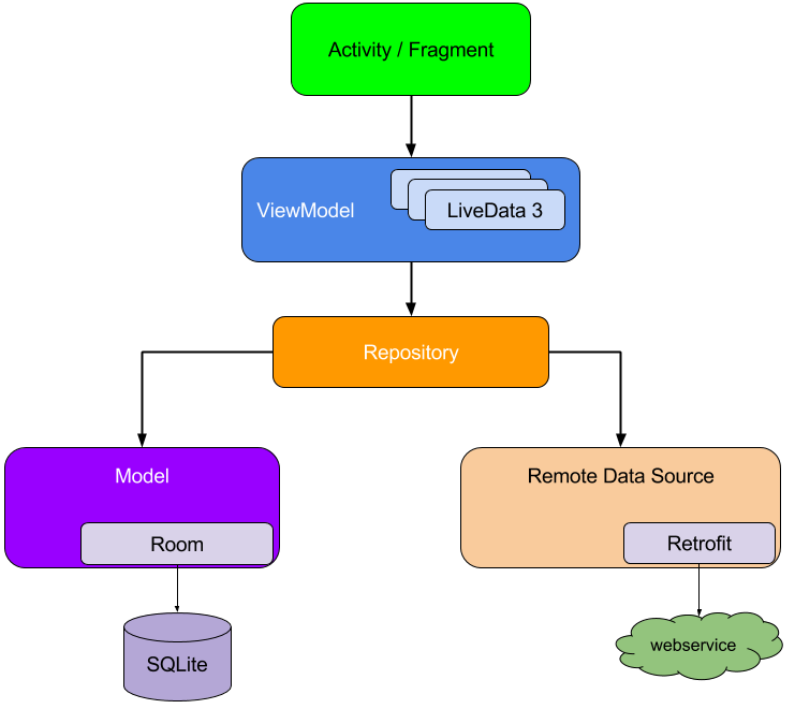
GDG

Manual DI implementation

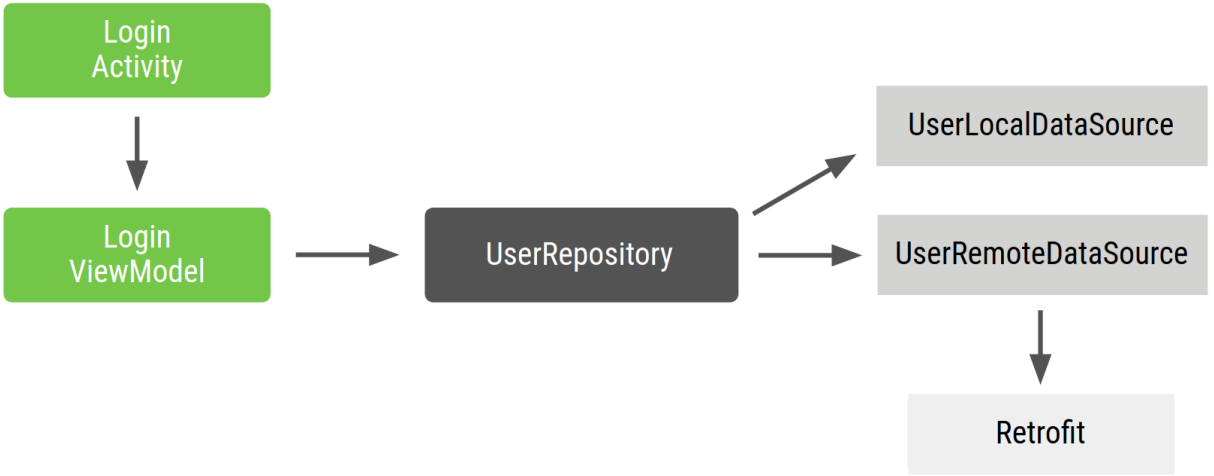


Manual DI implementation

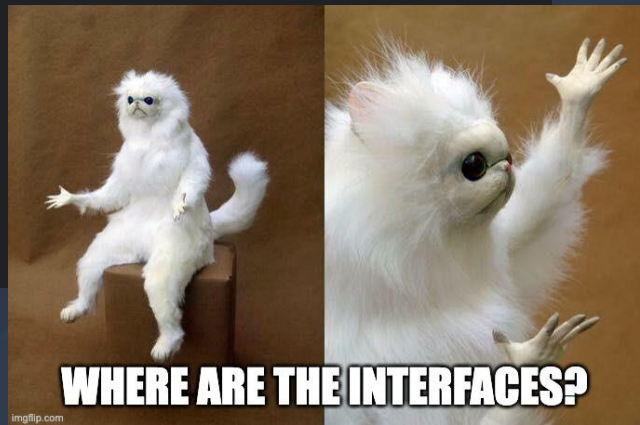
Manual DI implementation



Manual DI implementation



```
class LoginViewModel(
    private val userRepository: UserRepository
) { ... }

class UserRepository(
    private val localDataSource: UserLocalDataSource,
    private val remoteDataSource: UserRemoteDataSource
) { ... }

class UserLocalDataSource { ... }
class UserRemoteDataSource(
    private val loginService: ExternalLoginService
) { ... }
```



```
class LoginActivity: Activity() {

    private lateinit var loginViewModel: LoginViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val loginService: ExternalLoginService()
        val remoteDataSource = UserRemoteDataSource(loginService)
        val localDataSource = UserLocalDataSource()
        val userRepository = UserRepository(localDataSource,
                                            remoteDataSource)
        loginViewModel = LoginViewModel(userRepository)
    }
}
```

Why this sucks?

- **Lot of boilerplate code** – duplication of the same code pattern for each feature or screen
- **Dependencies have to be declared in order** – you can't instantiate a ViewModel object before Repository object
- **Difficult to reuse object** – to share Repository across multiple features you need to construct the Repository following the singleton pattern (deprecated) + makes testing more difficult because all tests share the same singleton instance

There are ways to improve it

- Manage dependencies with a **dependency container**
- Implement **factory pattern** for instantiation of multiple objects of the same type – example: you need a viewmodel in more places in the app

- Use an **DI framework** to simplify your development

```kotlin
interface Factory<T> {
    fun create(): T
}

class LoginViewModelFactory(private val userRepository: UserRepository) : Factory {
    override fun create(): LoginViewModel = LoginViewModel(userRepository)
}

class AppContainer {
    private val loginService = ExternalLoginService()
    private val remoteDataSource = UserRemoteDataSource(loginService)
    private val localDataSource = UserLocalDataSource()
    private val userRepository = UserRepository(localDataSource, remoteDataSource)
    // only VM factory gets exposed
    val loginViewModelFactory = LoginViewModelFactory(userRepository)
}
```

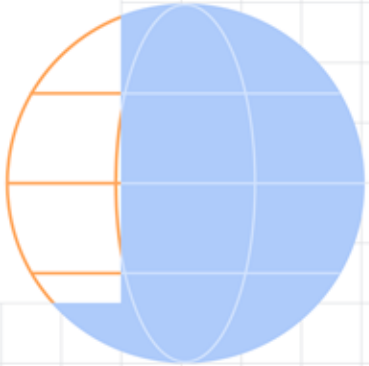
```kotlin
// Custom Application class that needs to be specified in the AndroidManifest.xml file
class MyApplication : Application() {
    val appContainer = AppContainer()
}

class LoginActivity: Activity() {

    private lateinit var loginViewModel: LoginViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val appContainer = (application as MyApplication).appContainer
        loginViewModel = appContainer.loginViewModelFactory.create()
    }
}
```

Drawbacks for this approach

- AppContainer quickly gets complicated when you want to include more functionality into your app
- Optimizing the AppContainer can be difficult and tiring – deleting or updating the whole dependency chain
- Resulting into **harder maintenance**, **reduced testability**, larger time consumption on **boilerplate code**, **error-prone**, **limited flexibility**, *depression and lack of motivation for developing*

GDG

Intro to Koin



Intro to Koin

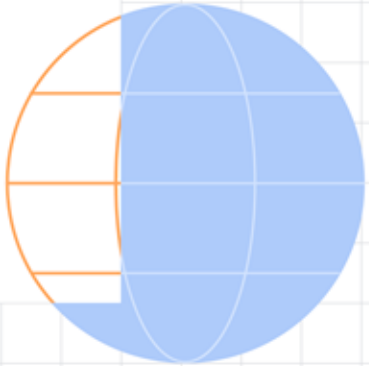
- DI framework which uses Kotlin DSL (Domain Specific Language) with **declarative approach**
- Popular because of its **simplicity** and support for Kotlin Multiplatform
- Latest stable version: v4.2
- Similar to service locator pattern (registry of available services ready to be requested for usage) but there are differences:
  - Koin defines its own **modules** instead of a **single static registry**
  - Dependencies are **tightly coupled** to the locator, Koin uses **loose coupling** due to modules and constructor injection

Koin terminology

- Module - a **container** for defining dependencies and their relationships
- Definition - a **declaration of a dependency** within a module. Definitions specify how to create or retrieve an instance of a particular type
- Scope - a mechanism for **controlling the lifecycle** of dependencies
  - single – singleton dependency (one instance across the application)
  - factory - a factory dependency, meaning a new instance is created each time it's requested
  - viewModel - a ViewModelScope instance for managing the lifecycle of coroutines within a ViewModel
- get() – resolves dependencies **inside module**
- inject() – retrieving dependency instances **from a module**
- Qualifier – **name of the definition**, differentiation between definitions of the same type

GDG

Koin implementation



Add Koin to project

```
dependencies {
    // ...
    val koinVersion = ”4.1.1”

    // Koin core features
    implementation("org.koin:koin-core:$koinVersion")

    // Koin Android features
    implementation("org.koin:koin-android:$koinVersion")
    implementation("org.koin:koin-androidx-viewmodel:$koinVersion")
    // ...
}
```

Define modules and dependencies

```
val networkingModule = module {
    single<ProfileApi>(qualifier = named("PrimaryProfileApi")) { PrimaryProfileApiImpl() }
    single<ProfileApi>(qualifier = named("OtherProfileApi")) { OtherProfileApiImpl() }
    //...
}
val repositoryModule = module {
    single<ProfileRepository> { ProfileRepositoryImpl(api = get(qualifier = named("PrimaryProfileApi"))) }
    //...
}
val viewModelModule = module {
    viewModel<ProfileViewModel> { (profileId) ->
        ProfileViewModelImpl(repository = get(), profileId = profileId)
    }
    //...
}
```

Initialize Koin

```
class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidContext(this@MyApplication)
            modules(networkingModule, repositoryModule, viewModelModule)
        }
    }
}

// Inside Manifest.xml
<manifest>
    <application
        android:name=".MyApplication" … />
</manifest>
```

Components injection

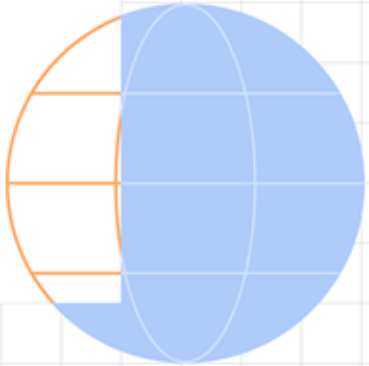
```
// Injection into Fragments or other classes
val vm by viewModel<ProfileViewModel> { parametersOf(profileId) }
val someService by inject<ProfileTranslator>()

// Injection into Composables
@Composable
fun App() {
    val vm = koinViewModel<ProfileViewModel>() // lazy initialization is not possible in composables
    val someService = koinInject<ProfileTranslator>()

    //…
}

@Composable
fun App(vm : ProfileViewModel = koinViewModel(), someService: ProfileTranslator = koinInject()) {
    //…
}
```

Practical example

Waiting for Android Studio to load up

GDG

Koin additional features



Autowire DSL

- No need to specify dependencies in constructor with **get()** function
- If you need to retrieve a “named” definition, you need to use the classic DSL with lambda and **get()** to specify the qualifier

```
class ClassA(val b : ClassB, val c : ClassC)
class ClassB ()
class ClassC ()

val autowiredModule = module {
   singleOf(::ClassA)
   singleOf(::ClassB)
   factoryOf(::ClassC)
}
```

```
module {
   singleOf(::ClassA) {
      // definition options
      named("my_qualifier")
      bind<InterfaceA>()
      createdAtStart()
   }
}
```

Lazy modules

- Enable asynchronous, parallel module loading to improve startup performance – instead of loading all modules synchronously at startup, you can defer and parallelize module initialization
- Distinct dependencies into basic modules and lazy modules (don't mix them up)

```
val lazyModule = lazyModule {
    includes(otherLazyModule)
    singleOf(::ClassA)
    factoryOf(::ClassB)
}

startKoin {
        // Load critical modules immediately
        modules(coreModule)
        // Load non-critical modules in background
        lazyModules(lazyModule)
}
```

Koin Component

- Provides an API to retrieve instances from the Koin container outside of module definitions
- Useful for classes that cannot use constructor injection (e.g., Activities or framework classes)
- With implementing **KoinComponent** interface, class gains access to Koin functions to resolve dependencies

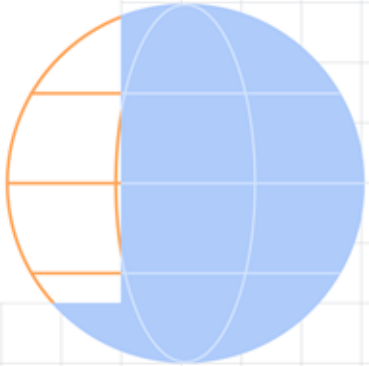
```
class MyComponent : KoinComponent {
    // Lazy evaluation - instance retrieved when first accessed
    val myService: MyService by inject()

    // or

    // Eager evaluation - instance retrieved immediately
    val myService: MyService = get()
}
```

Advanced patterns

- Managing collection of dependencies - e.g., on app started initialization tasks
- Optional dependencies - in some build environments (variants) we don't need particular dependencies, e.g., logging analytics for development environment
- Conditional bindings – depending on your build variant or feature flag you want to expose particular service to client code
- Type Aliases and Generics – for improving type clarity/readability and registering and injecting generic types like Repository<User> vs Repository<Product>

GDG

Best practices & common pitfalls

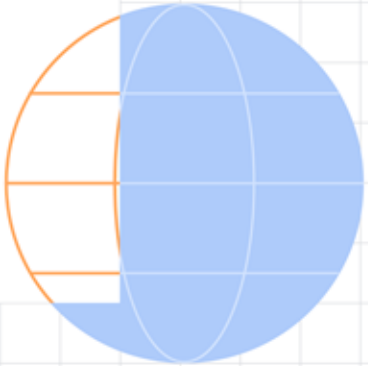


Best practices & common pitfalls

- Best practices:
  - **Separate responsibilities** – organize dependencies into layered modules (feature or functionality based)
  - Prefer **constructor injection**
  - Use **lazy injection** for **optional** dependencies
  - Write clear **unidirectional dependencies** – avoid circular references
  - Use property injection with **lazy injection** technique where possible
- Common pitfalls:
  - Incorrect scoping - overuse of Singletons
  - Tight Coupling
  - God modules
  - Unclear Module Structure
  - Depend on Activity contexts – memory leak

GDG

Alternatives



Alternatives

- **Dagger** – relies on compile-time code generation, processes annotations during compilation to generate the DI code for object instantiation and injection

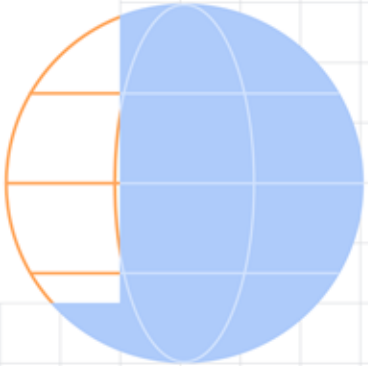
- **Hilt** – built on top of Dagger with more simplified and Android-oriented (pre-defined) annotations

Alternatives comparison

Feature	Koin	Dagger	Hilt
Approach	Runtime service locator	Compile-time code generation	Inherits from Dagger
Learning curve	Easy	Steep	Moderate
Boilerplate code	Minimal	High	Medium
Setup	Simple, declarative	Complex (requires component definitions)	Simplified setup than Dagger (predefined components and scopes)
Compile time	None	Increases (can be significant)	Increases (less than Dagger)
Debugging	Easier	Complex (code generation)	Easier than Dagger

GDG

Conclusion



Conclusion

- Dependency Injection promotes loose coupling, testability, and maintainability in your app
- Relying on interfaces makes your code clean 🪥
- Smaller or bigger project, Koin does the job 💪
- If you are not familiar with SOLID principles, check it out 👀

Google Developers

GDG

Thanks! :]

Questions?

Martin Zagoršćak
Android & KM dev @ Endava
@martin.zagorscak

Task & Homework

Implement DI system into your existing projects by following best practices.

@Home:

- Create a Logger class with a tag value (use Android's logger or external lib)
- Use (inject) it in key classes such as view models, repositories, api classes etc. to provide additional info for easier debugging